Universitat de Girona
Escola Politècnica Superior

GEINF

Pràctica 1: Estructures de Dades
Juguem amb Ce Essa Ves?
Xino-xano

Estructures de Dades i Algorítmica

Curs 2025/26

Universitat de Girona
Departament d'Informàtica,
Matemàtica Aplicada i Estadística

Hugo Heredia Sánchez
u6103255@campus.udg.edu

JUSTIFICACIÓ DE LES ESTRUCTURES DE DADES SELECCIONADES

En primer lloc, explicaré quines estructures de dades he seleccionat per emmagatzemar la informació necessària en cadascuna de les opcions del programa, així com la justificació de la seva elecció. Per tal de desar les diferents distàncies, he optat per un vector de set posicions —una per a cada distància—, ja que l'accés a qualsevol posició és té cost constant amortitzat, i el cost de recórrer-lo completament també pot considerar-se constant, atès que la mida (7) és molt reduïda. Per la mateixa raó, també he utilitzat un vector per emmagatzemar els transports associats a cada distància. Malgrat que en aquest cas el nombre màxim de transports podria arribar a ser d'uns trenta, aquesta N continua sent prou petita per considerar l'accés i manipulació dins del vector com a cost constant.

Per emmagatzemar les persones que utilitzen cada transport també he triat un vector, ja que l'accés a una posició concreta i la inserció al final tenen cost constant. No obstant això, el recorregut complet d'aquest vector sí que resulta lineal, degut a la quantitat elevada de persones. Finalment, per guardar els trajectes realitzats per cada persona he decidit tornar a utilitzar un vector, per les mateixes consideracions d'eficiència comentades anteriorment.

Pel que fa a l'opció c), he considerat que l'objecte **NumPersonesTransport** havia de ser una llista de pair<int, string>, de manera que per a cada tipus de transport s'hi pogués guardar el nombre de persones que l'utilitzen. Vaig optar per una llista perquè, mentre que ordenar un vector requeriria un doble bucle ($O(n^2)$), la llista permet aplicar el seu mètode intern sort, amb un cost O(n log n). Tanmateix, l'operació d'inserció té cost constant en ambdues estructures. A més, si tenim present que el nombre màxim de transports no és gaire elevat, la N pot considerar-se petita i, per tant, el cost d'ordenació pot aproximar-se a constant.

En l'opció d), l'objecte **MesRapids** està implementat com un vector de pair<string,double>, on per a cada distància es desa el nom del transport més ràpid i el seu temps mitjà. He triat un vector perquè la inserció té cost constant i, en ser només set posicions, el cost de recorregut es pot considerar també constant.

Per a l'opció e), he precalculat el trajecte més llarg durant el procés de lectura i emmagatzematge de dades. D'aquesta manera, evito haver de comprovar posteriorment quin trajecte és el més llarg per a cada persona, transport i distància, estalviant així un temps significatiu i evitant costos lineals al recórrer tots els vectors.

En l'opció f), l'objecte **NousTrajectes** és un vector de set<string>, on cada string representa la concatenació de la comarca d'inici i la comarca final separades per " - ". Cada set conté les diferents comarques que hi ha en una distància concreta. De nou, he triat un vector perquè la inserció és constant i, en tenir només set posicions, el seu recorregut implica un cost constant. El set garanteix que no hi hagi duplicats i manté els elements ordenats alfabèticament. No he fet servir un map perquè el propi string ja actua com a clau, sense necessitat d'emmagatzemar cap element. En el mètode afegir de la classe **Distancies** precalculo aquests set, guardant-hi les comarques corresponents a mesura que s'afegeixen els trajectes, evitant així haver de recórrer tots els vectors del programa i reduint significativament el temps.

A l'opció g), en el mètode afegir de la classe **Transports** també precalculo el nombre de trajectes associats a cada parella comarcaIn–comarcaFi, així com el temps total dels trajectes. Si no ho fes, hauria de recórrer totes les persones i tots els trajectes cada vegada que es cridés el mètode corresponent, cosa que

comportaria un cost lineal respecte al nombre de persones. Per emmagatzemar aquestes dades he triat un map<string, pair<int, int>>, fent servir com a clau la concatenació de les dues comarques. Així asseguro l'ordenació alfabètica i l'absència de duplicats.

A l'opció h), he tornat a precalcular en el mètode afegir de la classe **Transports** el nombre de trajectes i el temps total, però en aquest cas distribuïts per hores. Sense aquesta precalculació, caldria recórrer totes les persones i trajectes en cada crida i el cost seria lineal. Per guardar aquestes dades he utilitzat un map on l'element associat a cada clau és un vector de 24 posicions on es guarda un pair<int, int>, on cada posició representa una hora (0…23). El vector permet inserir en la posició corresponent amb cost constant. El map, com abans, garanteix l'ordenació alfabètica i l'absència de repeticions.

Per a l'opció j), l'objecte **MesOptim** és un struct que conté el tipus de transport, l'hora d'inici i el temps mitjà del trajecte. He considerat adequat utilitzar aquesta estructura perquè només cal retornar la informació d'un únic trajecte, i per tant no és necessari cap contenidor més complex.

Pel que fa a l'opció k), l'objecte **TempsPerNivellEstudis** és un map<string, double>, que permet emmagatzemar els nivells d'estudis com a claus ordenades alfabèticament i sense repeticions. L'element associat a cada clau és el temps mitjà corresponent dels trajectes.

Finalment, en l'opció m), en el mètode de lectura de dades de la classe **Mobilitat**, he precalculat els minuts que triga la persona que camina més estona per anar a treballar. D'aquesta manera s'evita haver de recórrer, per a cada distància, tots els transports, les persones i els trajectes per determinar quin és el trajecte més llarg d'aquest tipus, la qual cosa suposaria un cost considerablement més elevat.

JOCS DE PROVES

1. L'objectiu del primer joc de proves, anomenat t1.txt, és comprovar que el programa no doni cap error ni es tanqui quan l'usuari entri una opció que no existeix (no sigui cap de les tretze opcions disponibles).

2. L'objectiu del segon joc de proves, anomenat t2.txt, és comprovar que les dades que poguessin estar carregades prèviament s'eliminen a l'hora d'executar diferents arxius en una mateixa execució del programa.

3. L'objectiu del tercer joc de proves, anomenat t3.txt, és verificar que, quan un fitxer no s'obre correctament o s'introdueixen opcions abans d'haver-ne obert cap, els diferents mètodes retornen els valors esperats (0, "no trobat", un valor per defecte, etc.) i que el programa no es bloqueja ni es tanca inesperadament.

4. L'objectiu del quart joc de proves, anomenat t4.txt, és comprovar que les opcions 07, 08, 09 i 10 funcionen correctament i retornen els valors esperats

(0, “no trobat”, un valor per defecte, etc.) quan s’introdueixen kilòmetres negatius.

5. L’objectiu del cinquè joc de proves, anomenat t5.txt, és comprovar que l’opció 09 funciona correctament i retorna el valor esperat (0, “no trobat”, un valor per defecte, etc.) quan s’introdueixen hores negatives.